

Skin Disease Classification using YOLOv8

A Deep Learning Pipeline for Nine-Class Dermatological Lesion Classification from Dermoscopic Images

Shashank Choudhary

Roll Number: 25B0371

Mentor: Viren Mehta

Seasons of Code, IIT Bombay, AI for Healthcare

End Term Submission

1. Introduction and Problem Statement

Skin cancers and other dermatological lesions are common in both primary and specialist care. Early, accurate diagnosis matters most for melanoma and the keratinocyte carcinomas, where delay is linked to worse outcomes. Dermatologist review remains the clinical standard, but specialist access is limited, and even experienced clinicians disagree on ambiguous cases.

For this project, I set out to build an automated pipeline that takes a single dermoscopic image of a skin lesion and predicts which of nine common conditions it most likely represents, along with a confidence score. My goal was to build a decision-support tool, not a replacement for clinical judgement.

The nine target classes I worked with are Actinic Keratosis, Basal Cell Carcinoma, Dermatofibroma, Melanoma, Nevus, Pigmented Benign Keratosis, Seborrheic Keratosis, Squamous Cell Carcinoma, and Vascular Lesion. These span malignant conditions, one premalignant condition, and several benign but visually similar mimics, which is what makes this a realistic and clinically motivated problem to try and solve.

2. Proposed Solution and Methodology

2.1 Dataset

I used Roboflow's "skin diseases" classification set, published by user marblerace (<https://universe.roboflow.com/marblerace/skin-diseases-ewn74>). It contains 4,084 dermoscopic images sourced from the ISIC archive, identifiable from the retained ISIC_XXXXXXX.jpg filenames, labelled into the nine classes above under a CC BY 4.0 licence. I chose this dataset because ISIC is a widely used, clinically sourced benchmark in dermatology AI research, which let me check my results against prior work.

I accessed the data programmatically through the Roboflow Python SDK using a workspace-scoped API key, and downloaded it in "Folder Structure" format, one directory per class, which is what Ultralytics' YOLOv8 classifier expects.

2.2 Dataset Files and Updates

Before training, I checked the class-wise image counts in Roboflow's default split, and found **it was not stratified by class**:

Split	Classes present	Problem
train	9 / 9	None
valid	7 / 9	Basal Cell Carcinoma and Dermatofibroma missing entirely
test	3 / 9	6 classes had zero test images

Table 1. Class coverage in Roboflow's default split, before correction.

This is the part of the project I'd most like to highlight to you, since it directly affects how trustworthy my reported numbers are. Here is what I found, what I changed, and why:

- **Starting point:** 4,084 images across 9 classes, using Roboflow's default, unstratified train/valid/test split.
- **Problem I found:** the test split covered only 3 of 9 classes, and the valid split was missing 2 classes entirely, so 6 of 9 classes had no measurable test accuracy at all.
- **Change I made:** I pooled all images per class across the three default splits, then re-partitioned them using a stratified 70/15/15 split (scikit-learn's `train_test_split`, seed 42, applied independently per class).
- **Result:** 2,854 training, 613 validation, and 617 test images, with all 9 classes represented in every split. I verified this programmatically before running any training (Section 4, cells 3 to 4 of the notebook).

2.3 Model and Training Setup

I framed the task as single-label image classification rather than object detection, since each image carries one diagnostic label with no bounding box annotation. I used Ultralytics YOLOv8's classification variant (yolov8*-cls), initialised from COCO/ImageNet pretrained weights in all runs (transfer learning), since my training set was fairly modest in size.

I carried out three training runs on a Tesla T4 GPU on Google Colab:

Run	Backbone	Image size	Patience	Epochs run	Wall time
Run 1, baseline	yolov8s-cls	224	20	~44	~0.35 hr
Run 2, extended patience	yolov8s-cls	224	40	97	0.74 hr
Run 3, larger backbone	yolov8m-cls	224	30	54	0.43 hr

Table 2. Training configuration for all three runs. Batch size 32 throughout; other hyperparameters left at Ultralytics defaults.

Reason for three runs: I ran three configurations, not one, so that I could separate two possible causes behind a plateauing accuracy: insufficient training time, which Run 2's longer patience addresses, versus insufficient model capacity, which Run 3's larger backbone addresses. I discuss what I found in Section 3.

3. Results

3.1 Headline Accuracy Across Runs

I evaluated all three models on the same held-out, stratified test split (617 images, all 9 classes), using the best validation checkpoint from each run (best.pt), not the final epoch checkpoint.

Run	Best epoch	Test Top-1 Acc.	Test Top-5 Acc.
Run 1, yolov8s-cls, patience 20	~24	75.2%	98.2%
Run 2, yolov8s-cls, patience 40	57	75.7%	97.7%
Run 3, yolov8m-cls, patience 30	24	75.9%	97.7%

Table 3. Test set Top-1 and Top-5 accuracy across all three runs.

Result: all three of my runs plateau at roughly 75 to 76% top-1 accuracy. Extending patience from 20 to 40 epochs (Run 1 to Run 2) added only 0.5 points, and moving to a substantially larger backbone (Run 2 to Run 3, roughly 3 times the parameters) added only a further 0.2 points.

Reason: in both Run 2 and Run 3, the best checkpoint was reached well before training stopped (epoch 57 of 97, and epoch 24 of 54), which rules out “needed more epochs” as the explanation for the plateau. This tells me I am looking at a real difficulty ceiling in the classification problem itself, rather than a training-time or model-capacity limit, which I explore next through the confusion matrix.

3.2 Training Curves

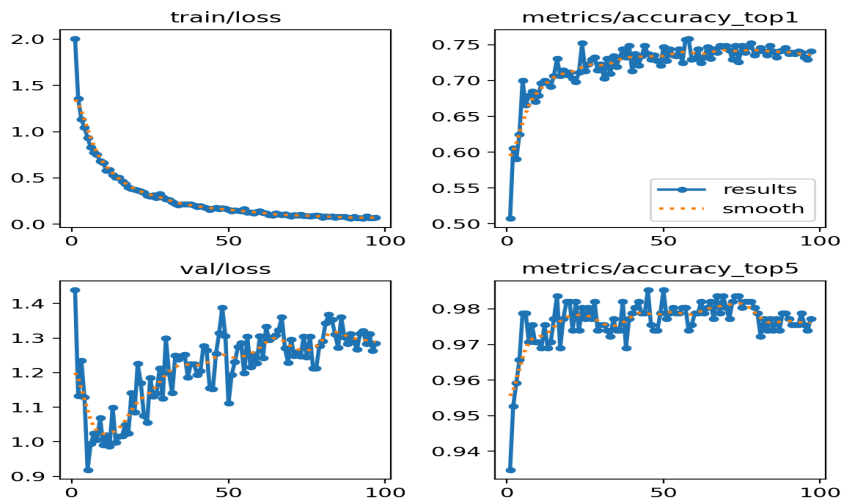


Figure 1. Loss and accuracy curves, Run 2 (yolov8s-cls, patience 40). Validation top-1 accuracy plateaus around epoch 55 to 60.

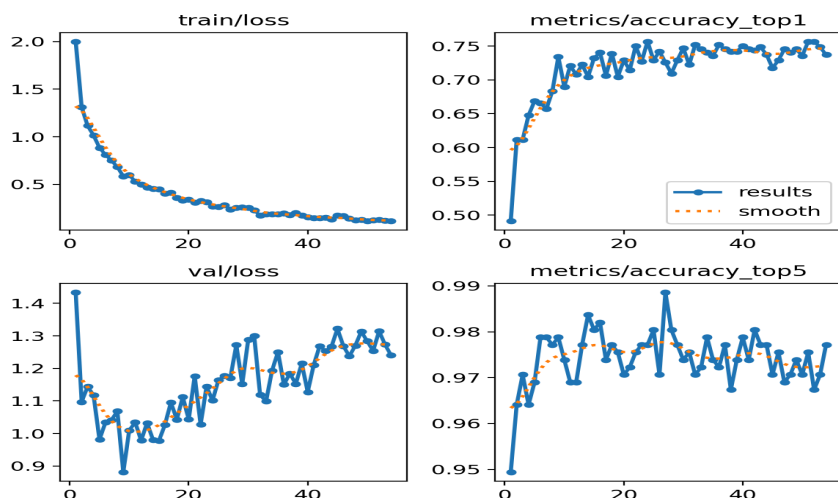


Figure 2. Loss and accuracy curves, Run 3 (yolov8m-cls, patience 30). The larger backbone converges faster (best epoch 24 vs. 57) but reaches the same ceiling.

3.3 Confusion Matrix Analysis

The confusion matrix for Run 2 (Figure 3) shows that my model's errors are concentrated in a specific, clinically explicable cluster, rather than scattered randomly.

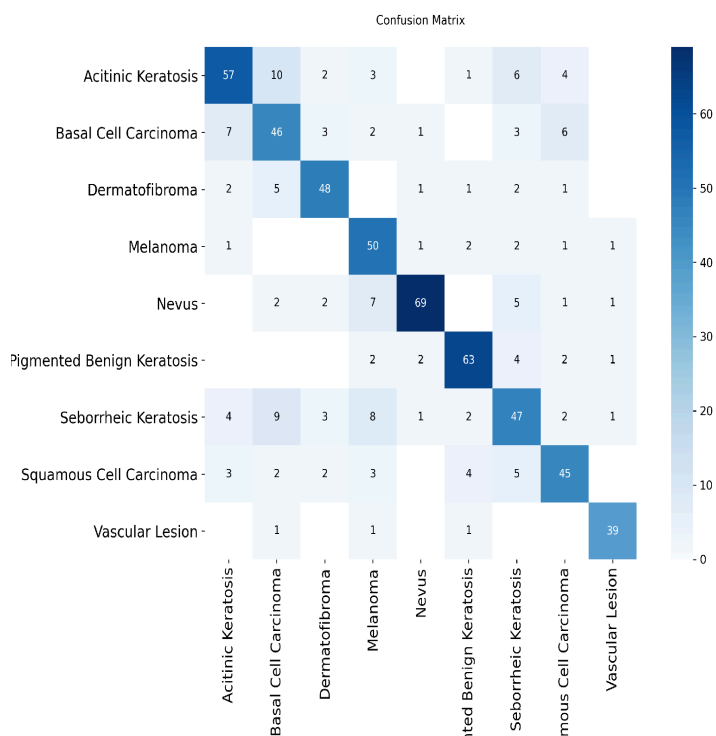


Figure 3. Confusion matrix, Run 2 (yolov8s-cls). Rows are true label, columns are predicted label.

- Actinic Keratosis (57 correct) is most often confused with Basal Cell Carcinoma (10), and to a lesser extent Seborrheic Keratosis (6).

- Basal Cell Carcinoma (46 correct) is confused with Actinic Keratosis (7) and Squamous Cell Carcinoma (6).
- Seborrheic Keratosis (47 correct) is confused with Basal Cell Carcinoma (9) and Squamous Cell Carcinoma (8).
- Squamous Cell Carcinoma (45 correct) spreads its errors across Seborrheic Keratosis (5), Vascular Lesion (5), and several other classes.
- By contrast, Nevus (69 correct) and Pigmented Benign Keratosis (63 correct) are my model's strongest classes, with very few off-diagonal errors.

Reason for the plateau: this cluster of confusions among Actinic Keratosis, Basal Cell Carcinoma, Squamous Cell Carcinoma, and Seborrheic Keratosis matches real dermatological difficulty — these are keratinocyte-associated lesions with overlapping texture, colour, and border features, hard to separate from single 2D images even for trained clinicians. Run 3's confusion matrix shows the same cluster persisting despite the larger backbone (see the notebook output), which supports a genuine visual-similarity ceiling in the data rather than a fixable modelling issue.

I think this is the most honest and important finding of my results: my model is highly reliable on visually distinct classes (Nevus, Pigmented Benign Keratosis) and systematically less reliable on one identifiable cluster of keratinocytic lesions. I see this as a limitation worth reporting clearly rather than smoothing over.

3.4 Qualitative Predictions

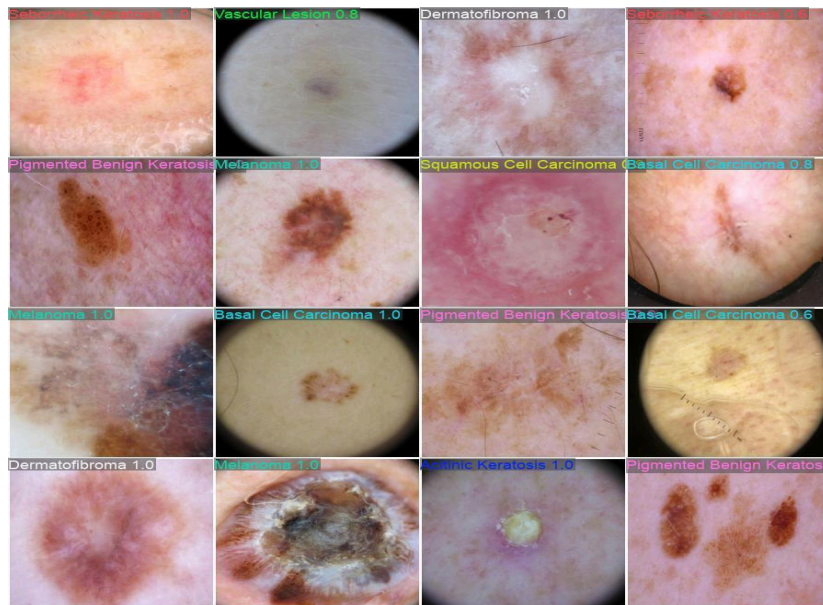


Figure 4. Sample predicted labels on validation images, Run 3 (yolov8m-cls).

4. Code Overview and Reproducibility

I implemented the full pipeline in a single Colab notebook:

- Cells 1 to 2: install dependencies (ultralytics, roboflow) and download the dataset via the Roboflow SDK using a workspace API key.
- Cell 2: inspect per-class image counts in the default split (this is where I surfaced the missing-class problem described in Section 2.2).

- Cells 3 to 4: re-partition the dataset into a class-stratified 70/15/15 split using scikit-learn's `train_test_split`, applied per class, then re-verify class coverage.
- Cell 5: train Run 1 and Run 2, yolov8s-cls, image size 224, batch 32, patience 20 then 40.
- Cell 6: train Run 3, yolov8m-cls, same data, image size 224, batch 32, patience 30.
- Each run produces `results.csv`, `results.png`, `confusion_matrix.png`, `confusion_matrix_normalized.png`, and a `weights/best.pt` checkpoint chosen by best validation accuracy.

Note: all the test-set metrics I report come from a `model.val(data=..., split="test")` call against each run's best.pt checkpoint, not the training-time validation numbers, so that I don't overstate performance.

5. Limitations and Future Work

- **Accuracy ceiling:** all three of my configurations converge to 75 to 76% top-1 test accuracy. Errors concentrate in the Actinic Keratosis, Basal Cell Carcinoma, Squamous Cell Carcinoma, Seborrheic Keratosis cluster. As future work, I would like to target this cluster with a class-weighted loss, more training data for the weakest classes, or a specialised secondary classifier for just this subset.
- **Single-image, single-label classification only:** my pipeline classifies one lesion per image with no localisation, and does not use the dermoscopic structure features that clinicians rely on for the hardest cases.
- **Class imbalance in the source data:** the classes were not equally sized to begin with (roughly 500 Basal Cell Carcinoma images versus roughly 200 Vascular Lesion images). I did not separately correct for this, and it likely contributes to weaker performance on the smaller classes.
- **No external validation:** I have only evaluated my model on a held-out split of the same dataset it was trained on. Its performance on a different source, camera, or population is untested.
- **No calibration beyond the dataset's existing labels:** I treated the ISIC labels as ground truth, with no additional clinician review of my specific test split.

6. Conclusion

In this project, I built and evaluated a nine-class skin lesion classifier using YOLOv8, trained on an ISIC-derived Roboflow dataset.

A meaningful part of my work was diagnostic: I found that the default split omitted entire classes from validation and test. I caught this before training and corrected it with a proper class-stratified re-split, without which my reported accuracy would not have been trustworthy.

I compared three configurations to rule out insufficient training time and insufficient model capacity as causes of the plateau (75.2% to 75.7% to 75.9% top-1). **The confusion matrix shows why:** the remaining errors concentrate in one clinically coherent cluster of keratinocyte-related lesions, while my model is highly accurate on visually distinct classes like Nevus and Pigmented Benign Keratosis.

My honest bottom line is a real, defensible ceiling of about 76% top-1 and 98% top-5 accuracy on a genuinely hard 9-class medical imaging task, with an identified and explicable source for the remaining errors. The natural next step, in my view, is a targeted intervention on the AK/BCC/SCC/Seborrheic Keratosis confusion cluster, rather than further generic hyperparameter tuning.

References

[1] marblerace. skin diseases Classification Dataset. Roboflow Universe, 2025.
<https://universe.roboflow.com/marblerace/skin-diseases-ewn74>

[2] International Skin Imaging Collaboration (ISIC) Archive. <https://www.isic-archive.com/>

[3] Ultralytics YOLOv8 Documentation. <https://docs.ultralytics.com/>

[4] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pages 2825 to 2830, 2011. (train_test_split used for stratified re-splitting)